

Web Scrapping and Data Collection Using Python

Scrapping Data from Webpages and Storing in CSV/JSON



by

Dr M Rajasekhara Babu

School of
Computer Science and Engineering
<https://www.rajababu.org>

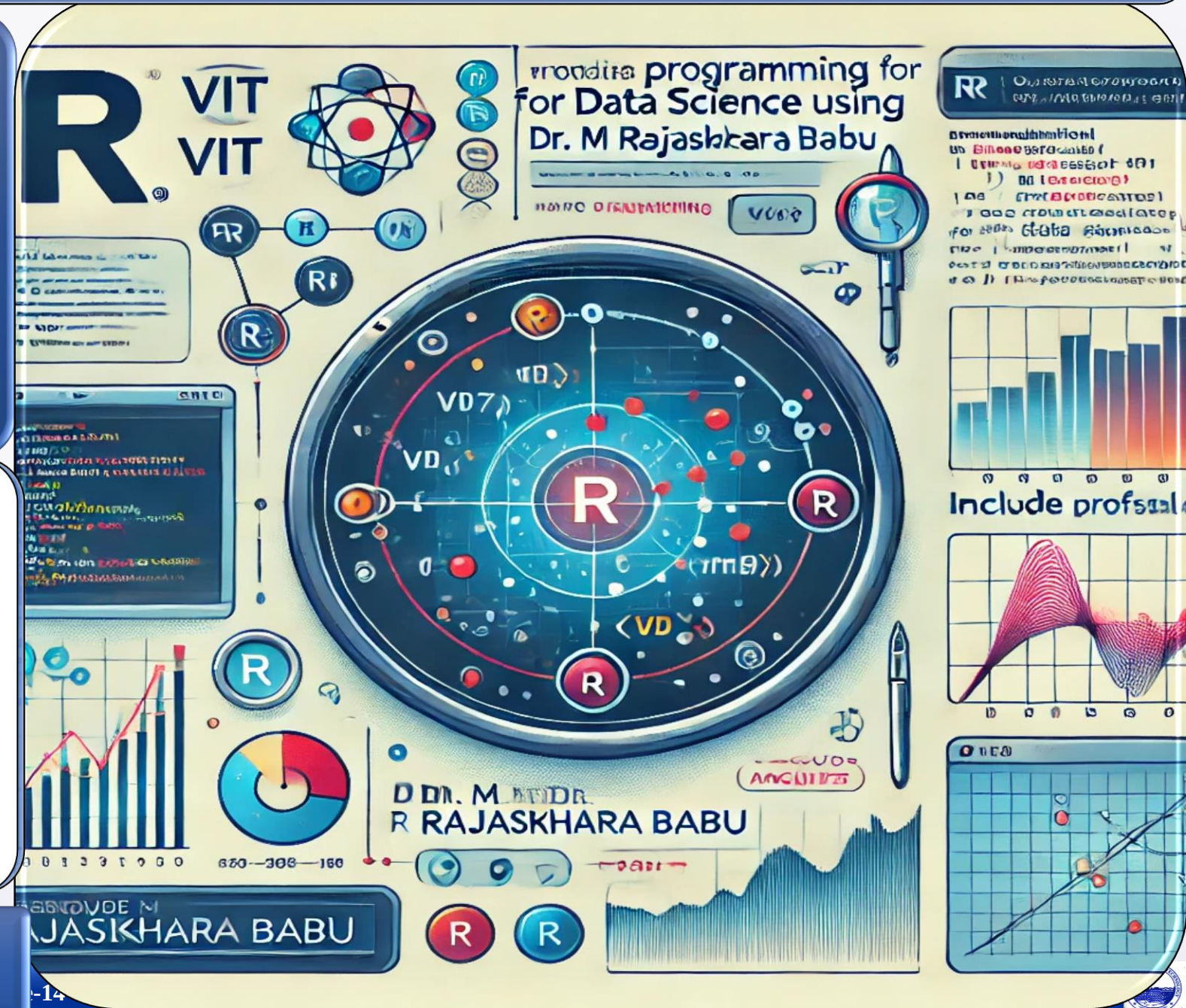


VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Vellore-632014, Tamil Nadu, India



What is Web Scraping?

Web Scraping is the automated process of extracting information from websites.

Market Research

Price Monitoring

News Aggregation

Social Media Analytics

Data Science Projects

Web Scraping Architecture

Website

HTTP Request

HTML Page

Parser

The scraping pipeline begins with an HTTP request to a website, retrieves the HTML page, parses it with BeautifulSoup, extracts the target data, and finally stores it in CSV or JSON format.

Scrape data from a webpage and store it in a structured format like CSV or JSON

Import Statements

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import json
```

Library	Purpose
requests	Access webpages
BeautifulSoup	Parse HTML
pandas	Store structured data
json	JSON file handling

GET Request

```
import requests

url = "https://quotes.toscrape.com"
response = requests.get(url)
```

Purpose

- The `requests.get()` method sends an HTTP GET request to the specified URL and retrieves the full webpage content from the server.
- The `response` object contains the HTML source that will be passed to the parser in the next step.

Parsing HTML with BeautifulSoup

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(
    response.text,
    "html.parser"
)
```

Role of BeautifulSoup

- **Parses HTML**
Converts raw HTML text into a navigable tree structure.
- **Searches Tags**
Locates specific HTML elements by tag name or class.
- **Extracts Information**
Pulls text and attributes from matched elements.

Sample HTML Structure

HTML tags contain the information to be extracted. Understanding the structure is essential before writing any scraping logic.

```
<div class="quote">  
  <span class="text">"Quote Text"</span>  
  <small class="author">Author</small>  
</div>
```

i Inspect the webpage's HTML using browser DevTools (F12) to identify the correct tags and class names before writing your scraper.

Extraction Code

```
for quote in soup.find_all(
    "div",
    class_="quote"):

    text = quote.find(
        "span",
        class_="text"
    ).get_text()

    author = quote.find(
        "small",
        class_="author"
    ).get_text()
```

How It Works

- `find_all()` returns a list of all matching HTML elements.
- For each `div` with class `quote`, we locate the nested `span` and `small` tags and call `.`
- `get_text()` to retrieve the visible text content — the quote and its author.

Building the Lists

```
quotes = []  
authors = []  
  
quotes.append(text)  
authors.append(author)
```

Purpose

- Python lists act as temporary containers for scraped data during the loop.
- Each iteration appends one quote and one author, keeping the two lists aligned by index.
- Once the loop completes, these lists are passed into a pandas DataFrame for structured storage and export.

Creating a DataFrame

```
df = pd.DataFrame({  
    "Quote": quotes,  
    "Author": authors  
})
```

Example Output

Quote	Author
"The world as we have created it..."	Albert Einstein
"It is our choices that show..."	J.K. Rowling
"There are only two ways to live..."	Albert Einstein

A pandas DataFrame organizes scraped data into labeled rows and columns, making it easy to inspect, filter, and export.

CSV Export Code

```
df.to_csv(  
    "quotes.csv",  
    index=False  
)
```

Output File

quotes.csv

Why CSV?

- CSV (Comma-Separated Values) is one of the most widely used formats in data science.
- Setting `index=False` prevents pandas from writing the row index as an extra column.
- The resulting file can be opened in Excel, Google Sheets, or loaded directly into any data analysis pipeline.

JSON Export Code

```
df.to_json(  
    "quotes.json",  
    orient="records",  
    indent=4  
)
```

Output Structure

```
[  
  {  
    "Quote": "Example quote  
text",  
    "Author": "Author Name"  
  },  
  {  
    "Quote": "Another  
quote",  
    "Author": "Another  
Author"  
  }  
]
```

Using `orient="records"` produces a list of objects — one per row — which is ideal for web APIs and JavaScript applications.

CSV

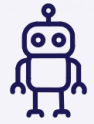
- Tabular, row-column structure
- Lightweight and compact
- Ideal for data analysis tools
- Easy to open in spreadsheets

JSON

- Hierarchical, nested structure
- Flexible and expressive
- Preferred for web applications
- Machine-readable and API-friendly

Choose **CSV** for tabular analysis workflows and **JSON** when data needs to integrate with web services or APIs.

Benefits of Web Scraping



Automated Collection

Eliminates manual copy-paste, saving hours of effort.



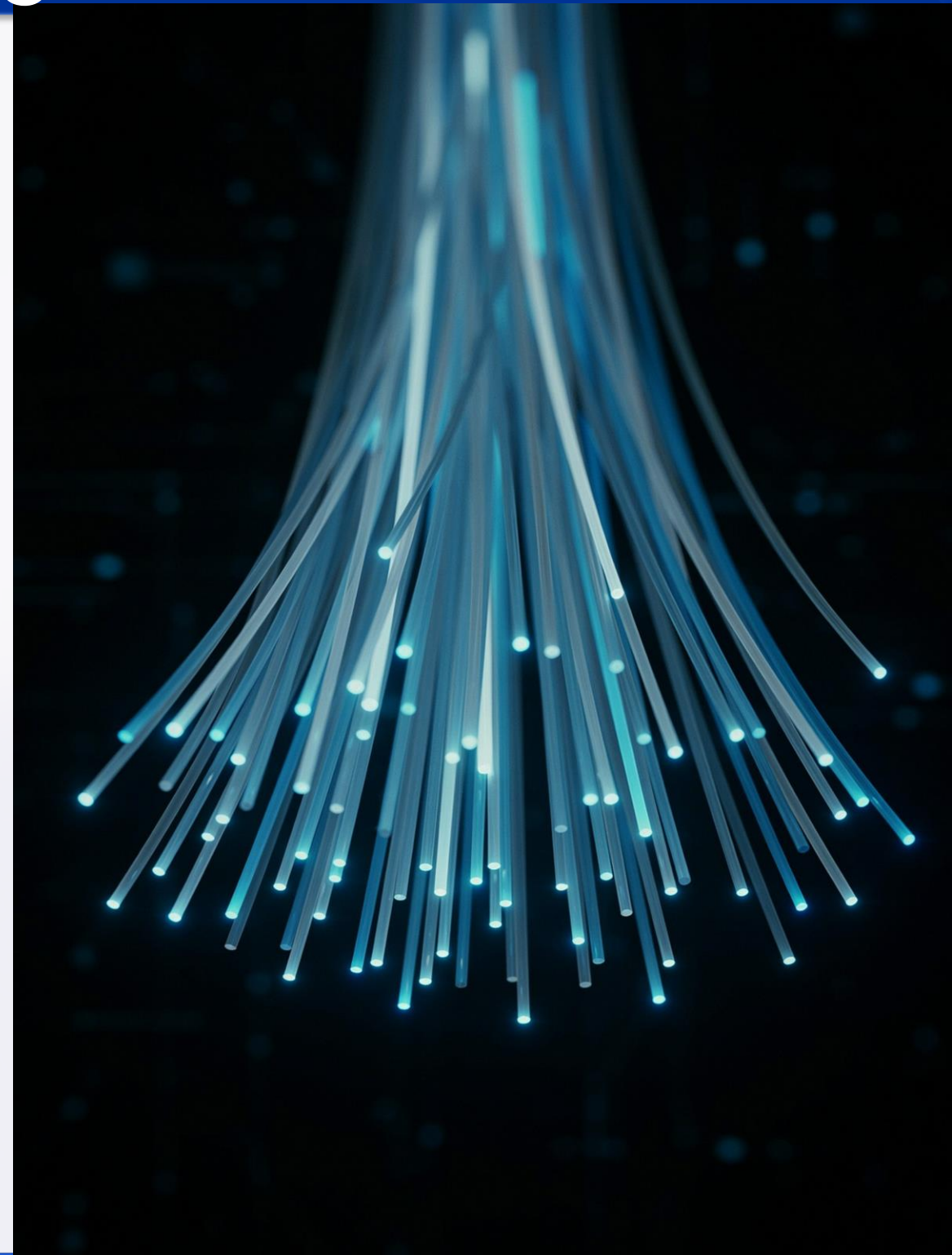
Real-Time Data

Captures live information as it changes on the web.



Supports Analytics

Feeds directly into data science and BI pipelines.



Challenges in Web Scraping

Website Structure Changes

HTML layouts can change without notice, breaking existing scrapers.

CAPTCHA Protection

Sites use CAPTCHAs to block automated bots from accessing content.

Dynamic Websites

JavaScript-rendered pages require tools beyond basic requests.

Legal Restrictions

Always review a site's Terms of Service and robots.txt before scraping.

Rate Limits

Sending too many requests too quickly can result in IP bans.



Business Analytics

Scrape product prices and customer reviews for competitive intelligence.

Finance

Collect stock market data and financial indicators in real time.

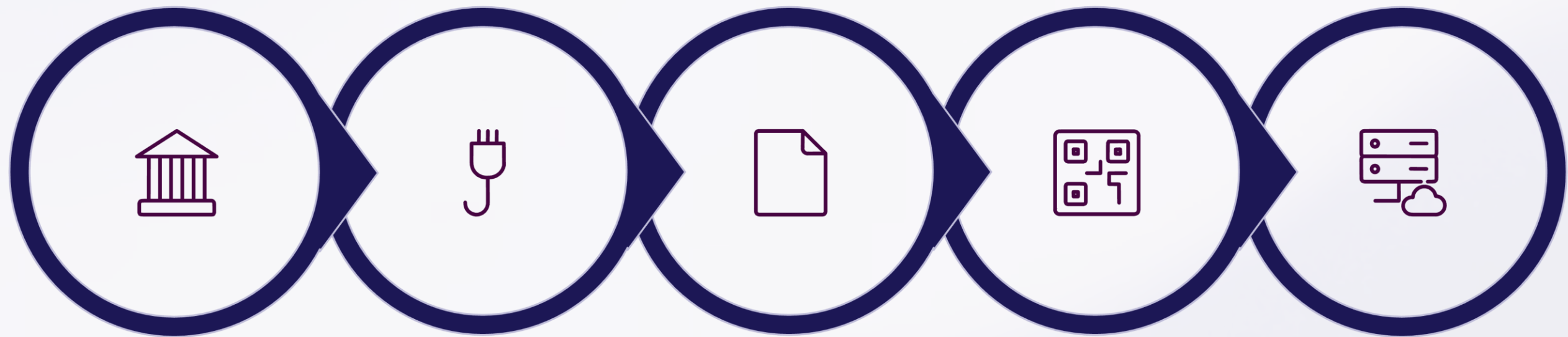
Research

Gather academic publications and articles for literature analysis.

Social Media

Perform trend analysis by extracting posts and engagement metrics.

Complete Program Flow



**Import
Libraries**

**Send HTTP
Request**

Receive HTML

Parse HTML

Extract Data

This end-to-end pipeline represents a complete, reusable web scraping program. Each step builds on the previous one, from importing libraries all the way to persisting structured data in CSV or JSON format.

Write a Python program to scrape and store book data from a website.

1 Scrape Book Titles

Use `requests` and `BeautifulSoup` to extract book titles from a website.

2 Store in a DataFrame

Organize the extracted titles into a pandas DataFrame with appropriate column labels.

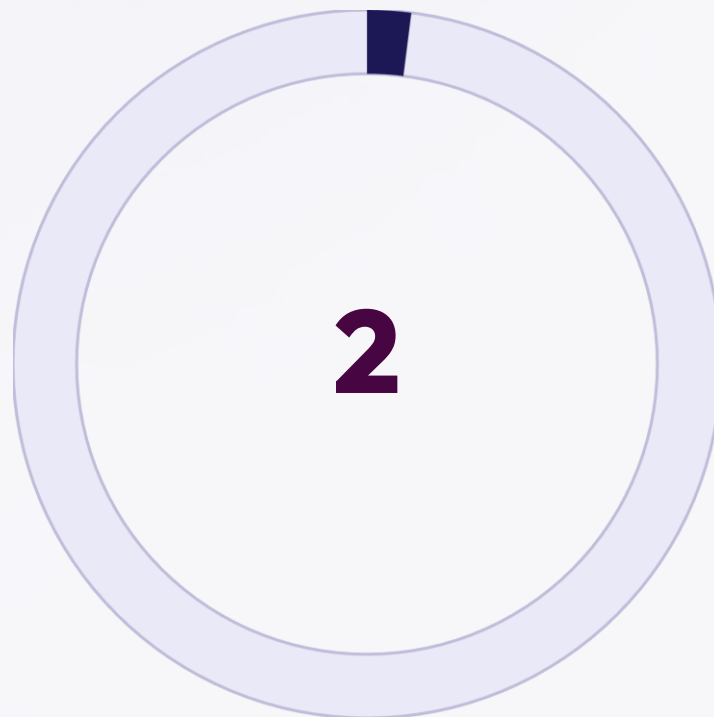
3 Save as CSV

Export the DataFrame to a `.csv` file using `df.to_csv()`.

4 Save as JSON

Export the same DataFrame to a `.json` file using `df.to_json()`.

Summary & Key Takeaways



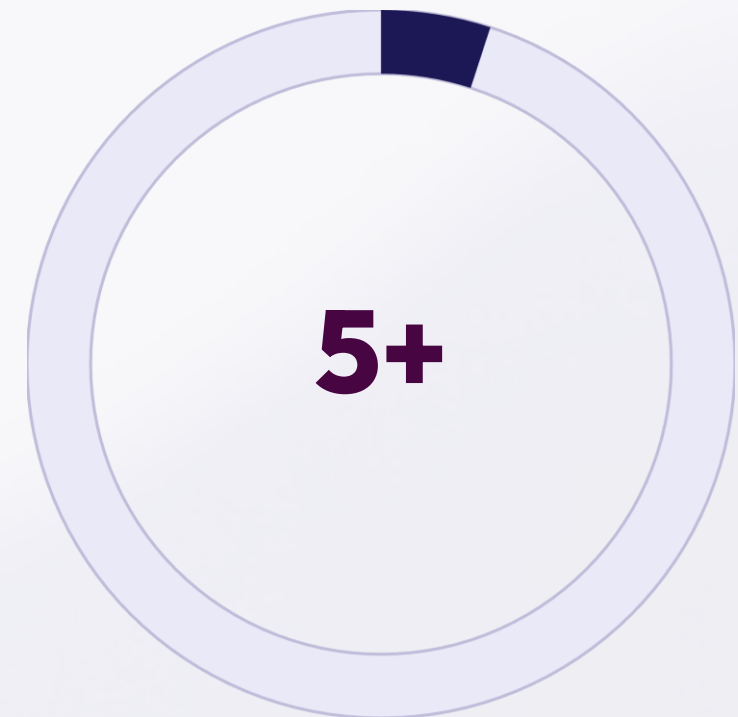
Core Libraries

Requests & BeautifulSoup power the scraping pipeline.



Output Formats

CSV for analysis, JSON for web and API integration.



Real-World Uses

Finance, research, social media, business analytics, and more.

Students can now scrape data from webpages, parse HTML using BeautifulSoup, collect structured web data, and store results in CSV and JSON — a complete data collection technique for Data Science projects.

- **Introduction to Feature Engineering**
 - Concept and importance of feature engineering
 - Benefits of creating transformed features
 - Improving model performance through feature transformation
- **Introduction to Exploratory Data Analysis (EDA)**
 - Purpose and objectives of EDA
 - Understanding distributions, patterns, and relationships
 - Common EDA techniques
- **Dataset Creation and Preparation**
 - Using Pandas, NumPy, and Matplotlib
 - Generating random datasets
 - Understanding features and target variables
 - Importance of reproducibility using random seeds
- **Feature Transformation Techniques**
 - Logarithmic transformation
 - Feature engineering using derived variables
 - Benefits of reducing skewness and improving distributions
- **Feature Binning**
 - Converting continuous values into categories
 - Equal-width and equal-frequency binning
 - Creating Low, Medium, and High categories using `pd.cut()`
- **Correlation Analysis**
 - Understanding correlation coefficients
 - Computing correlation matrices
 - Interpreting relationships among variables
- **Data Visualization**
 - Scatter plot creation using Matplotlib
 - Color mapping using target variables
 - Pattern recognition and cluster analysis
- **Pattern Interpretation and Applications**
 - Identifying positive and negative correlations
 - Understanding clusters and overlaps
 - Real-world applications in banking, healthcare, retail, and AI